

CLOUD COMPUTING

LAB NO # 14



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

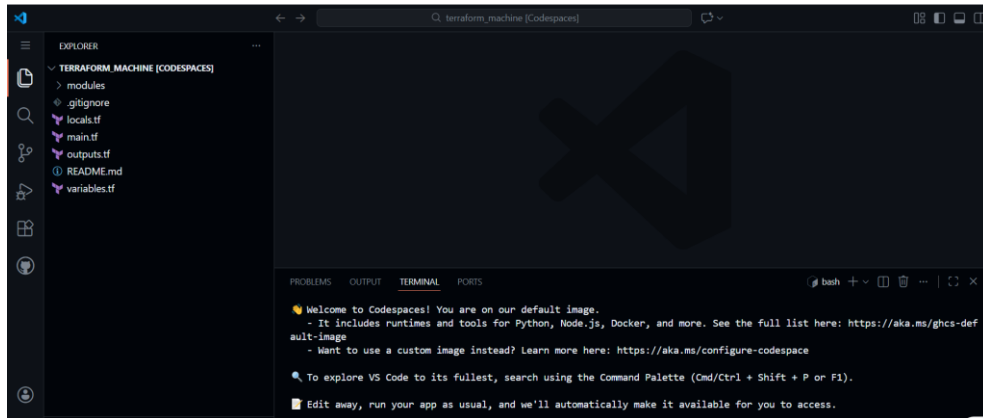
Semester: V-A

CLOUD COMPUTING

LAB NO # 14

Task 0 – Lab Setup (Codespace & GH CLI)

Create and open a GitHub Codespace from the forked repository.



Verify installed tools inside the Codespace terminal: `Aws --version`

```
aws-cli/2.32.7 Python/3.12.12 Linux/6.8.0-1030-azure source/x86_64.alpine.3
```

Terraform `--version`

```
Terraform v1.7.0
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.14.4. You can update by downloading from https://www.terraform.io/downloads.html
```

Ansible `--version` || `echo "ansible not yet installed"`

```
ot yet installed"
ansible [core 2.20.0]
  config file = None
  configured module search path = ['/home/vscode/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.12/site-packages/ansible
  ansible collection location = /home/vscode/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.12 (main, Oct 11 2025, 01:16:26) [GCC 15.2.0] (/usr/bin/python3)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

Verify AWS CLI authentication and permissions:

Aws `aws sts get-caller-identity`

```
{
  "UserId": "AIDA5WPGPNRI7ANGZM5NF",
  "Account": "941618064465",
  "Arn": "arn:aws:iam:941618064465:user/Admin"
}
```

Task 1 – Generate SSH Key and Initial Terraform Apply

Check existing SSH keys and generate a new SSH key pair if required:

Before key generation:

Ls `~/ssh`

```
ls: cannot access '/home/vscode/.ssh': No such file or directory
```

Key generation: `ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""`

```
Generating public/private ed25519 key pair.
Created directory '/home/vscode/.ssh'.
Your identification has been saved in /home/vscode/.ssh/id_ed25519
Your public key has been saved in /home/vscode/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:f+2ojaUlnqXjngMrLFLGL6jan/Yq3h8FJgAnK89aqe6A  vscode@codespaces-079caa
The key's randomart image is:
+--[ED25519 256]--+
|
|..
|+oo. S
|+*... ..
|oo+ *o.o . o .
|Eo=B.BB.. * o
|oBX*B+= .+.o .
+----[SHA256]-----+
```

After key generation:

`Ls ~/.ssh`

```
total 20
drwx----- 2 vscode vscode 4096 Jan 28 16:17 .
drwxr-xr-x 1 vscode vscode 4096 Jan 28 16:17 ..
-rw----- 1 vscode vscode 419 Jan 28 16:17 id_ed25519
-rw-r--r-- 1 vscode vscode 106 Jan 28 16:17 id_ed25519.pub
```

Navigate to the project directory and create terraform.tfvars.

Touch terraform.tfvars

`Ls -la terraform.tfvars`

```
-rw-rw-rw- 1 vscode vscode 0 Jan 28 16:37 terraform.tfvars
```

Add required Terraform variables including VPC CIDR, subnet CIDR, instance type, and SSH keys.

```
lab_14 > Task > terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 availability_zone = "me-central-1a"
4 env_prefix = "dev"
5 instance_type = "t3.micro"
6 public_key = "~/.ssh/id_ed25519.pub"
7 private_key = "~/.ssh/id_ed25519"
```

Initialize Terraform using `terraform init`.

```
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Apply Terraform to create two EC2 instances:

```
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
webserver_public_ips = [  
  "3.28.74.36",  
  "158.252.33.79",  
]
```

Display Terraform outputs to retrieve EC2 public IPs.

Terraform output

```
webserver_public_ips = [  
  "3.28.74.36",  
  "158.252.33.79",  
]
```

Task 2 – Static Ansible Inventory with Two EC2 Instances

Install Ansible Core using pipx and verify installation: `Ansible --version`

```
ansible [core 2.20.0]  
config file = None  
configured module search path = ['/home/vscode/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']  
ansible python module location = /usr/lib/python3.12/site-packages/ansible  
ansible collection location = /home/vscode/.ansible/ansible/collections:/usr/share/ansible/collections  
executable location = /usr/bin/ansible  
python version = 3.12.12 (main, Oct 11 2025, 01:16:26) [GCC 15.2.0] (/usr/bin/python3)  
jinja version = 3.1.6  
pyyaml version = 6.0.3 (with libyaml v0.2.5)
```

Retrieve EC2 public IPs using Terraform output.

Terraform output

```
webserver_public_ips = [  
  "3.28.74.36",  
  "158.252.33.79",  
]
```

Create an Ansible inventory file named hosts.

Touch hosts

Ls -la hosts

```
-rw-rw-rw- 1 vscode vscode 0 Jan 28 17:51 hosts
```

Add both EC2 public IPs along with SSH user and private key details.

Cat hosts

```
3.28.74.36    ansible_user=ec2-user  ansible_ssh_private_key_file=~/.ssh/id_ed25519  
158.252.33.79  ansible_user=ec2-user  ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

Test connectivity using Ansible ping module.

`Cd /workspaces/CC-AminaNoor-007/lab_14/Task && chmod 755 . && ansible all -I hosts -m ping`

```

158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}

```

Disable strict host key checking if required and re-test connectivity.
 Ansible all -I hosts -m ping

```

.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}

```

Task 3 – Scale to Three Instances & Group-Based Inventory

Update the Terraform webserver module to use count = 3.

```

lab_14 > Task > main.tf
23 module "myapp-webserver" {
29   my_ip = local.my_ip
30   vpc_id = aws_vpc.myapp_vpc.id
31   subnet_id = module.myapp-subnet.subnet.id
32
33   # Loop count
34   count = 3
35   # Use count.index to differentiate instances
36   instance_suffix = count.index
37

```

Apply Terraform to provision three EC2 instances. Terraform apply -auto-approve

```

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

```

Outputs:

```

webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
  "40.172.100.131",
]

```

Verify outputs showing three public IPs.

Terraform output

```

webserver_public_ips = [
  "3.28.74.36",
  "158.252.33.79",
  "40.172.100.131",
]

```

Rewrite the Ansible inventory using group definitions [ec2] and [droplet].

Cat hosts

```
[ec2]
3.28.74.36
158.252.33.79

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[droplet]
40.172.100.131

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Test connectivity for individual groups, single hosts, and all instances.

Ansible ec2 -I hosts -m ping

```
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.33.79' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

Ansible 3.28.74.36 -I hosts -m ping

```
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

Ansible droplet -I hosts -m ping

```
40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

Ansible all -I hosts -m ping

```

> lab_12
> lab_13
> lab_14
  Task
    .terraform
    modules
    .terraform.lock... U
    ansible.cfg
    hosts
    locals.tf
    main.tf
    outputs.tf
    terraform.tfstate
    terraform.tfstate.ba...
    terraform.tfvars
    variables.tf
    README.md
  Lab_Exam
  LabProject_FrontendB...

158.252.33.79 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '40.172.100.131' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {

```

Task 4 – Global Ansible Configuration & First Nginx Playbook

Create a global Ansible configuration file at ~/.ansible.cfg.

```

[default]
host_key_checking = False
interpreter_python = /usr/bin/python3
~
~
~

```

Disable strict host key checking globally.

Cd /workspaces/CCAMinaNoor-007/lab_14/ && cat hosts

```

[ec2]
3.28.74.36
158.252.33.79

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[droplet]
40.172.100.131

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519

```

Remove SSH common arguments from inventory file.

Ansible all -I hosts -m ping

```

> lab_12
> lab_13
> lab_14
  Task
    .terraform
    modules
    .terraform.lock... U
    ansible.cfg
    hosts
    locals.tf
    main.tf
    outputs.tf
    terraform.tfstate
    terraform.tfstate.ba...
    terraform.tfvars
    variables.tf
    README.md
  Lab_Exam
  LabProject_FrontendB...

40.172.100.131 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.74.36' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.74.36 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.33.79' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.33.79 | SUCCESS => {
  "ansible_facts": {

```

Verify connectivity using Ansible ping.

Touch my-playbook.yaml
Ls -ls my-playbook.yaml

```
-rw-rw-rw- 1 vscode vscode 0 Jan 28 18:33 my-playbook.yaml
```

Create an Ansible playbook to install and start Nginx.

```
lab_14 > Task > ! my-playbook.yaml
1  ---
2  - name: Configure nginx web server
3    hosts: ec2
4    become: true
5    tasks:
6      - name: install nginx and update cache
7        yum:
8          name: nginx
9          state: present
10         update_cache: yes
11
12      - name: start nginx server
13        service:
14          name: nginx
```

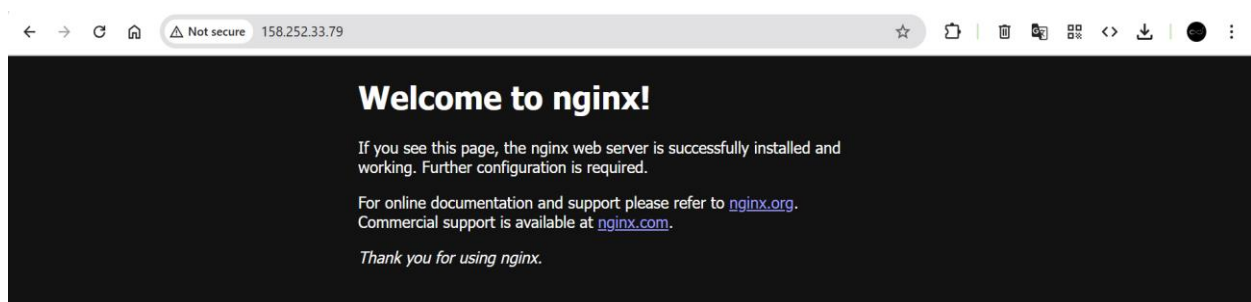
Execute the playbook on the [ec2] group.
Ansible-playbook -I hosts my-playbook.yaml

```
TASK [install nginx and update cache] *****
changed: [3.28.74.36]
changed: [158.252.33.79]

TASK [start nginx server] *****
changed: [158.252.33.79]
changed: [3.28.74.36]

PLAY RECAP *****
158.252.33.79      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
3.28.74.36       : ok=3    changed=2    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

Verify Nginx deployment using a web browser.



Update the playbook to target the [droplet] group and re-run.


```
lab_14 > Task > ! my-playbook.yaml U X
1 ---
2 - name: Configure nginx web server
3   hosts: droplet
4   become: true
5   tasks:
6     - name: install nginx and update cache
7       yum:
8         name: nginx
```

Ansible-playbook -I hosts my-playbook.yaml



Task 5 – Single Nginx Target Group & HTTPS Prerequisites

Touch ansible.cfg

```
-rw-rw-rw- 1 vscode vscode 55 Jan 28 18:46 ansible.cfg
```

Ls -la ansible.cfg

```
lab_14 > Task > ansible.cfg
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3
```

Update Terraform to reduce EC2 count to one instance.

```
lab_14 > Task > main.tf
23 module "myapp-webserver" {
30   vpc_id = aws_vpc.myapp_vpc.id
31   subnet_id = module.myapp-subnet.subnet.id
32
33   # Loop count
34   count = 1
35   # Use count.index to differentiate instances
36   instance_suffix = count.index
37 }
```

Apply Terraform changes and retrieve the public IP.
Terraform apply -auto-approve

Apply complete! Resources: 0 added, 0 changed, 6 destroyed.

Update the Ansible inventory to define a [nginx] group.
Terraform output

```
webserver_public_ips = [
  "3.28.74.36",
]
```

Update the Nginx playbook to install OpenSSL and enable services.
Cd /aorkspaces/CC-AminaNoor-007/lab_14 && cat hosts

```
[nginx]
3.28.74.36

[nginx:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_user=ec2-user
```

Run the playbook and verify Nginx using a browser.

```
lab_14 > Task > my-playbook.yaml
1 ---
2 - name: Configure nginx web server
3   hosts: nginx
4   become: true
5   tasks:
6     - name: install nginx and update cache
7       yum:
8         name: nginx
```

Ansible-playbook -I hosts my-playbook.yaml



Task 6 – Ansible-Managed SSL Certificates

Extend the existing playbook to include SSL certificate tasks.

```

lab_14 > Task > ! my-playbook.yaml U X  ansible.cfg U  main.tf U
2  - name: Configure nginx web server
22
23  - name: Configure SSL certificates
24    hosts: nginx
25    become: true
26    tasks:
27      - name: Create SSL private directory
28        file:
29          path: /etc/ssl/private
30          state: directory
31          mode: '0700'
32
33      - name: Create SSL certs directory
34        file:

```

Create required directories for SSL keys and certificates.

Ansible-playbook -I hosts my-playbook.yaml

```

TASK [Get IMDSv2 token] *****
ok: [3.28.74.36]

TASK [Get current public IP] *****
ok: [3.28.74.36]

TASK [Show current public IP] *****
ok: [3.28.74.36] => {
  "msg": "Public IP: 3.28.74.36"
}

TASK [Generate self-signed SSL certificate] *****
changed: [3.28.74.36]

PLAY RECAP *****
3.28.74.36 : ok=11 changed=3 unreachable=0 failed=0 skipped=0 resc
ued=0 ignored=0

```

Retrieve EC2 public IP dynamically using IMDSv2. Generate self-signed SSL certificates using OpenSSL. Verify SSL certificate and key files via SSH. Ssh [ec2-user@3.28.74.36](ssh://ec2-user@3.28.74.36) -I ~/.ssh/id_ed25519

```

-----BEGIN CERTIFICATE-----
MIIDOTCCAIGGAwIBAgIudHsr1NBx7xjvonqcIFzoyCz+nIowOQYJKoZIhvcNAQEL
BQAwFETETMBEGA1UEAwkMy4yOC43NC4zNjAeFw0yNjAxMjg3OTA3MzBaFw0yNjAx
Mjg3OTA3MzBaBMBuExARBgNVBAMMCjMuMjguNzQuMzYwggE1MA0GCQSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQCYS+o/3Yf87MvBGbnA8pN/FuyxitkkiCD6bhY5WVOA
AwVvKrPH28TBs3PDDO2aREM7yWi5hqZMUfdwkkT9GDznEK9dBedHCGmXiGtvoHGH
RNQw1fcwFRdBwz9gHdvQwSW0uMpyobK+hk88/mfElGiTr1hrX/cYBsyJrVA2Q2Ts
Ohv+9A3QLIrHYq4MPOAeYsQAk1kzNLccxao+
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAgEAAoIBAQCYS+o/3Yf87MvBGbnA8pN/
FuyxitkkiCD6bhY5WVOAaAwVvKrPH28TBs3PDDO2aREM7yWi5hqZMUfdwkkT9GDzn
EK9dBedHCGmXiGtvoHGHRNQw1fcwFRdBwz9gHdvQwSW0uMpyobK+hk88/mfElGiTr
1hrX/cYBsyJrVA2Q2TsOhv+9A3QLIrHYq4MPOAeYsQAk1kzNLccxao+
MTK+Hh1bXtRhH+LerYpVp3gDOU94wikO2RUymFIN12iatw3rAugbmg02VdIY7gs
-----

```

Task 7 – PHP Front-End Deployment with Templates

This task deploys a PHP web application and configures Nginx using templates.

Create required directories for files and templates.

Mkdir -p files templates

Touch files/index.php

Touch templates/nginx.conf.j2

Ls -R

```

.:
ansible.cfg  locals.tf  my-playbook.yaml  terraform.tfstate  variables.tf
files        main.tf    outputs.tf         terraform.tfstate.backup
hosts        modules   templates          terraform.tfvars

./files:
index.php

./modules:
subnet webserver

./modules/subnet:
main.tf  outputs.tf  variables.tf

./modules/webserver:
main.tf  outputs.tf  variables.tf

./templates:
nginx.conf.j2

```

Create a PHP metadata web page

```

lab_14 > Task > files > index.php
1  <?php
2  // Get hostname
3  $hostname = gethostname();
4
5  // Deployment date
6  $deployed_date = date("Y-m-d H:i:s");
7
8  // Metadata base URL
9  $metadata_base = "http://169.254.169.254/latest/";
10
11 // Function to get IMDSv2 token
12 function getImdsV2Token() {

```

Create an Nginx configuration template using Jinja2.

```

lab_14 > Task > templates > nginx.conf.j2
1  user nginx;
2  worker_processes auto;
3  error_log /var/log/nginx/error.log notice;
4  pid /run/nginx.pid;
5
6  events {
7      worker_connections 1024;
8  }
9
10 http {
11     log_format main '$remote_addr - $remote user [$time local] "$request"'

```

Update the playbook to deploy PHP, copy files, and restart services.

```

lab_14 > Task > ! my-playbook.yaml
23  - name: Configure SSL certificates
26      tasks:
73
74  - name: Deploy Nginx website and configuration files
75      hosts: nginx
76      become: true
77      tasks:
78          - name: install php-fpm and php-curl
79              yum:
80                  name:
81                      - php-fpm
82                      - php-curl

```

Run the playbook and verify the PHP application over HTTPS.

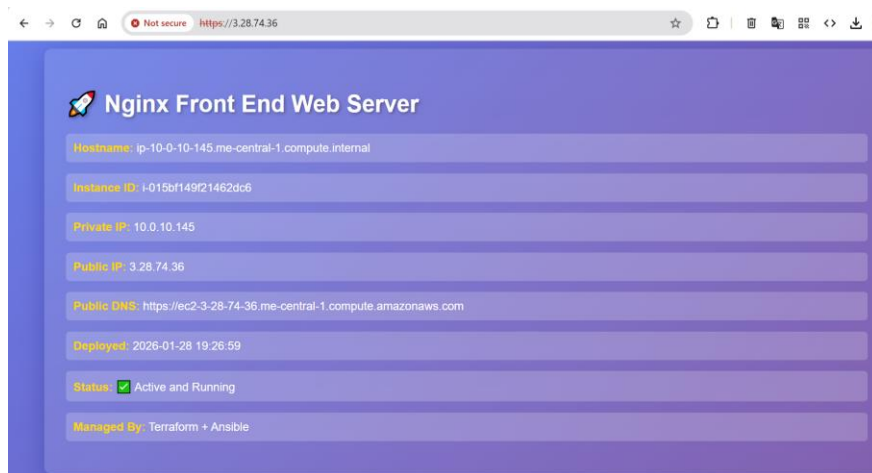
Cd /workspaces/CCaminaNoor/lab_14 && ansible-playbook -I hosts my-playbook.yaml

The top part of the image shows the output of an Ansible playbook execution. The tasks completed are:

- TASK [install php-fpm and php-curl] ok: [3.28.74.36]
- TASK [Copy website files] ok: [3.28.74.36]
- TASK [Copy nginx.conf template] changed: [3.28.74.36]
- TASK [Restart nginx] changed: [3.28.74.36]
- TASK [Start and enable php-fpm] changed: [3.28.74.36]

The PLAY RECAP shows 19 hosts ok, 3 changed, and 0 failed or skipped.

The bottom part of the image shows a Chrome browser security warning. The address bar shows "Not secure" and the URL "https://3.28.74.36". The warning message is "Your connection is not private" and "Attackers might be trying to steal your information from 3.28.74.36 (for example, passwords, messages, or credit cards).". There are buttons for "Advanced" and "Back to safety".



Task 8 – Docker & Docker Compose Provisioning via Ansible

Destroy existing infrastructure using Terraform.

Terraform destroy -auto-approve

```
Destroy complete! Resources: 7 destroyed.
```

Recreate a fresh EC2 instance.

Terraform apply -auto-approve

```
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

Terraform output

```
webservers_public_ips = [
  "40.172.234.49",
]
```

Update Ansible inventory for Docker servers.

```
lab_14 > Task > hosts
1  [docker_servers]
2  40.172.234.49
3
4  [docker_servers:vars]
5  ansible_ssh_private_key_file=~/.ssh/id_ed25519
6  ansible_user=ec2-user
```

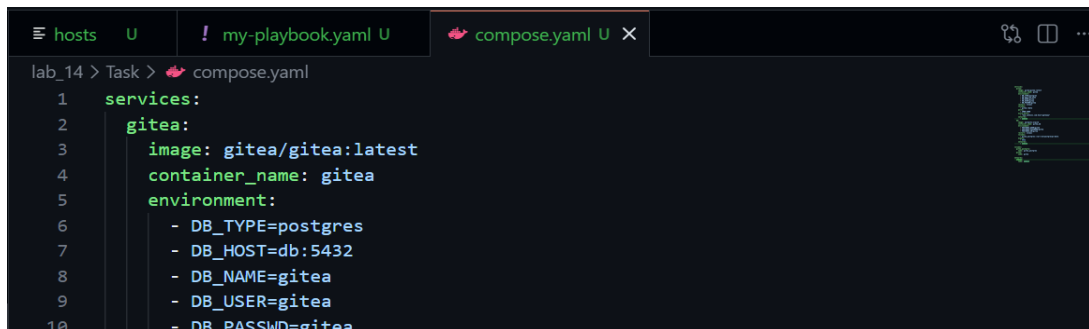
Replace the playbook to install Docker and Docker Compose.

```
lab_14 > Task > ! my-playbook.yaml
1  ---
2  - name: Configure Docker
3    hosts: all
4    become: true
5    tasks:
6      - name: install docker and update cache
7        yum:
```

Run the playbook and verify Docker installation via SSH.

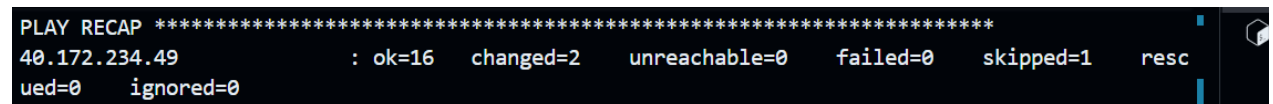
Ansible-playbook -I hosts my-playbook.yaml

Make compose.yaml and code.



```
lab_14 > Task > compose.yaml
1  services:
2    gitea:
3      image: gitea/gitea:latest
4      container_name: gitea
5      environment:
6        - DB_TYPE=postgres
7        - DB_HOST=db:5432
8        - DB_NAME=gitea
9        - DB_USER=gitea
10       - DB_PASSWD=gitea
```

Cd /workspaces/CC-AminaNoor/lab_14 && ansible-playbook -I hosts my-playbook.yaml 2>&1 | head -100



```
PLAY RECAP *****
40.172.234.49 : ok=16  changed=2  unreachable=0  failed=0  skipped=1  rescued=0  ignored=0
```

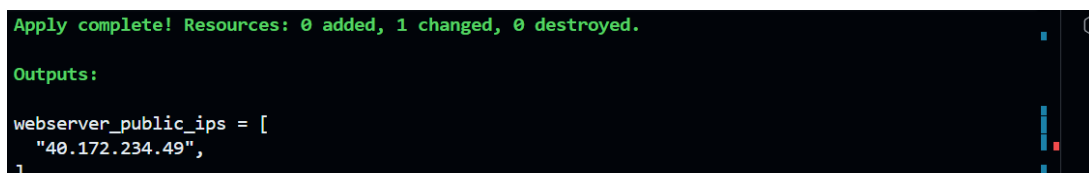
Update Terraform security group to allow port 3000.



```
lab_14 > Task > modules > webserver > main.tf
1  resource "aws_security_group" "web_sg" {
18    ingress {
23    }
24    ingress {
25      from_port = 3000
26      to_port   = 3000
27      protocol  = "tcp"
28      cidr_blocks = ["0.0.0.0/0"]

```

Cd /workspaces/CC-AminaNoor/lab_14 && terraform apply -auto-approve 2>&1 | tail -30



```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "40.172.234.49",
]

```

Apply Terraform changes and access Gitea via browser.

Initial Configuration

If you run GitLab inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

GitLab requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type * PostgreSQL

Host * db:5432

Username * gitlab

Password *

Database Name * gitlab

SSL * Disable

Schema

Leave blank for database default ("public").

Port number the GitLab web server will listen on.

GitLab Base URL * http://40.172.234.49:3000/

Base address for HTTPS clone URLs and email notifications.

Log Path * /data/gitlab/log

Log files will be written to this directory.

☐ Enable Update Checker

Checks for new version releases periodically by connecting to gitlab.io.

Optional Settings

► Email Settings

► Server and Third-Party Service Settings

► Administrator Account Settings

These configuration options will be written into: /data/gitlab/config/settings.yml

Install GitLab

Task 10 – Automating Ansible with Terraform (null_resource)

Update main.tf and add a null_resource that triggers Ansible using EC2 public IPs from Terraform outputs.

```

39 resource "null_resource" "configure_server" {
40   triggers = {
41     webserver_public_ips_for_ansible = join(",", [for i in module.myapp-webserver :
42   ]
43 }
44 depends_on = [module.myapp-webserver]
45
46 provisioner "local-exec" {
47   command = <<-EOT
48   ansible-playbook -i ${self.triggers.webserver_public_ips_for_ansible}, \

```

Destroy existing infrastructure to ensure a clean run.

Terraform destroy -auto-approve

```

aws_vpc.myapp_vpc: Destruction complete after 1s
Destroy complete! Resources: 7 destroyed.

```

Recreate infrastructure using Terraform and observe Ansible execution during apply.

Terraform apply -auto-approve

```

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "158.252.83.213",
]

```

Verify that the Gitea or Nginx application is reachable via browser after automation.

Initial Configuration

If you run Gitea inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

Gitea requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type * PostgreSQL

Host * db:5432

Username * gitea

Password *

Database Name * gitea

SSL * Disable

Schema

Leave blank for database default ("public").

Task 11 – Dynamic Inventory with aws_ec2 Plugin

Update ansible.cfg to enable the aws_ec2 inventory plugin and set required defaults.

```

lab_14 > Task > ansible.cfg
1  [defaults]
2  host_key_checking=False
3  interpreter_python = /usr/bin/python3
4  deprecation_warnings = False
5
6  enable_plugins = aws_ec2
7  private_key_file = ~/.ssh/id_ed25519

```

Create a new dynamic inventory file named inventory_aws_ec2.yaml.

Touch inventory_aws_ec2.yaml

Ls -la inventory_aws_ec2.yaml

```
-rw-rw-rw- 1 vscode vscode 0 Jan 28 20:19 inventory_aws_ec2.yaml
```

Add initial plugin configuration and AWS region details.

```

lab_14 > Task > ! inventory_aws_ec2.yaml
1  ---
2  plugin: aws_ec2
3  regions:
4  - me-central-1

```

Update Terraform configuration to create both **dev** and **prod** EC2 instances.

```

lab_14 > Task > main.tf
39 module "myapp-webserver-prod" {
40
41     # Loop count
42     count = 1
43     # Use count.index to differentiate instances
44     instance_suffix = count.index
45 }
46
47 resource "null_resource" "configure_server" {
48     triggers = {
49         webserver_public_ips_for_ansible = join(",", [for i in module.myapp-webserver :
50

```

Add Terraform outputs to display public IPs for both environments.

```

lab_14 > Task > outputs.tf
1  output "webserver_public_ips" {
2      value = [for i in module.myapp-webserver : i.aws_instance.public_ip]
3  }
4
5  output "prod-webserver_public_ips" {
6      value = [for i in module.myapp-webserver-prod : i.aws_instance.public_ip]
7  }

```

Terraform apply -auto-approve

```

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

```

Terraform output

```

prod-webserver_public_ips = [
    "3.28.137.90",
]
webserver_public_ips = [
    "158.252.83.213",
]

```

1. Install boto3 and botocore Python libraries required by the plugin.

Sudo apk add py3-boto3 py3-botocore && python3 -c “import boto3, botocore; print(f'boto3:{boto3.__version__}’)”

```

(1/6) Installing py3-botocore (1.40.75-r0)
(2/6) Installing py3-botocore-pyc (1.40.75-r0)
(3/6) Installing py3-s3transfer (0.13.0-r0)
(4/6) Installing py3-s3transfer-pyc (0.13.0-r0)
(5/6) Installing py3-boto3 (1.40.75-r0)
(6/6) Installing py3-boto3-pyc (1.40.75-r0)
OK: 1157.9 MiB in 285 packages
boto3: 1.40.75

```

Verify plugin functionality by generating the Ansible inventory graph.
Ansible-inventory -I inventory_Aws_ec2.yaml --graph

```

@all:
|--@ungrouped:
|--@aws_ec2:
|   |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|   |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com

```

Task 12 – Filtering EC2 Instances by Tags & Instance Type

Update inventory_aws_ec2.yaml to enable grouping based on instance tags.

```
3      26b91e901: "-"
8      b1e47x: f98
1      - key: f982
2      keyeq~8lonb2:
3      - ws-cenr19J-T
3      u8870u2:
5      b7n87u: 9m2~ec5
7      ---
lab_14 > j92k > i iul6uio1~9m2~ec5~9awu
E mo2f 0 i ul-b79pook~9awu 0 i iul6uio1~9m2~ec5~9awu X
```

Generate the Ansible inventory graph to verify tag-based grouping.
Ansible-inventory -I inventory-aws-ec2.yaml --graph

```
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@tag_Name_prod_ec2_instance_0:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@tag_Name_dev_ec2_instance_0:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
```

Extend the inventory configuration to group instances by instance type.

```
lab_14 > Task > ! inventory_aws_ec2.yaml
3      regions:
5
6      keyed_groups:
7      - key: tags
8        prefix: tag
9        separator: "_"
10
11     - key: instance_type
12       prefix: instance_type
13       separator: "_"
```

Verify updated inventory graph showing both tag and instance-type groups.: Ansible-inventory -I inventory-aws-ec2.yaml --graph

```
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@tag_Name_prod_ec2_instance_0:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@instance_type_t3_nano:
| |--ec2-3-28-137-90.me-central-1.compute.amazonaws.com
|--@tag_Name_dev_ec2_instance_0:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
|--@instance_type_t3_micro:
| |--ec2-158-252-83-213.me-central-1.compute.amazonaws.com
```

Update my-playbook.yaml to support execution on all hosts.

```
lab_14 > Task > ! my-playbook.yaml
1      ---
2      - name: Configure nginx web server
3        hosts: all
4        become: true
5        tasks:
6        - name: install nginx and update cache
7          yum:
8            name: nginx
9            state: present
10           update cache: yes
```

Run the playbook on all EC2 instances.: Ansible-playbook -I inventory_aws_ec2.yaml my-playbook.yaml

```
TASK [Restart nginx] *****
changed: [ec2-3-28-137-90.me-central-1.compute.amazonaws.com]
changed: [ec2-158-252-83-213.me-central-1.compute.amazonaws.com]

TASK [Start and enable php-fpm] *****
changed: [ec2-3-28-137-90.me-central-1.compute.amazonaws.com]
changed: [ec2-158-252-83-213.me-central-1.compute.amazonaws.com]

PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=10  unreachable=0  fa
iled=0  skipped=0  rescued=0  ignored=0
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=10  unreachable=0  faile
d=0  skipped=0  rescued=0  ignored=0
```

Execute the playbook selectively for dev, prod, and specific instance types.: Ansible-playbook -I inventory-aws-ec2.yaml -l tag_Name_dev_* my-playbook.yaml

```
PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  fa
iled=0  skipped=0  rescued=0  ignored=0
```

Ansible-playbook -I inventory-aws-ec2.yaml -l tag_Name_prod_* my-playbook.yaml

```
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  faile
d=0  skipped=0  rescued=0  ignored=0
```

Ansible-playbook -I inventory-aws-ec2.yaml -l instance_type_t3_micro my-playbook.yaml

```
PLAY RECAP *****
ec2-158-252-83-213.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  fa
iled=0  skipped=0  rescued=0  ignored=0
```

Ansible-playbook -I inventory-aws-ec2.yaml -l instance_type_t3_nano my-playbook.yaml

```
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  faile
d=0  skipped=0  rescued=0  ignored=0
```

Set the dynamic inventory as default in ansible.cfg.

```
lab_14 > Task > ansible.cfg
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3
4 deprecation_warnings = False
5 remote_user = ec2-user
6 private_key_file = ~/.ssh/id_ed25519
7 inventory = ./inventory_aws_ec2.yaml
```

Run Ansible without specifying the inventory explicitly: Ansible-playbook -l instance_type_t3_nano my-playbook.yaml

```
PLAY RECAP *****
ec2-3-28-137-90.me-central-1.compute.amazonaws.com : ok=18  changed=1  unreachable=0  faile
d=0  skipped=0  rescued=0  ignored=0
```

Task 13 – Ansible Roles: nginx, ssl, webapp

Update Terraform to deploy a single dev EC2 instance.

Cat main.tf

```
my_ip = local.my_ip
vpc_id = aws_vpc.myapp_vpc.id
subnet_id = module.myapp-subnet.subnet.id

# Loop count
count = 1
# Use count.index to differentiate instances
instance_suffix = count.index
```

Create a structured Ansible directory with inventory, roles, and configuration files.

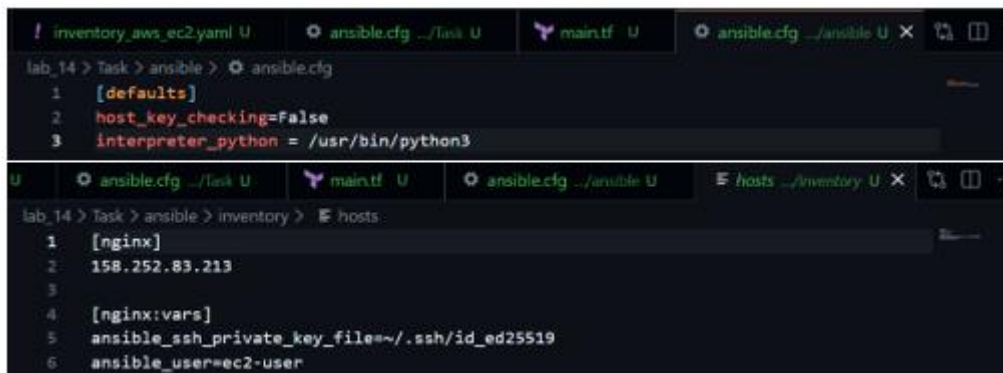
```
Mkdir -p ansible
Cd ansible
Mkdir inventory roles
Touch ansible.cfg my-playbook.yaml
Ls -R
```

```
.:
ansible.cfg inventory my-playbook.yaml roles

./inventory:

./roles:
```

Configure project-level ansible.cfg and static inventory.



The screenshot shows a code editor with two files open. The top file is `ansible.cfg` with the following content:

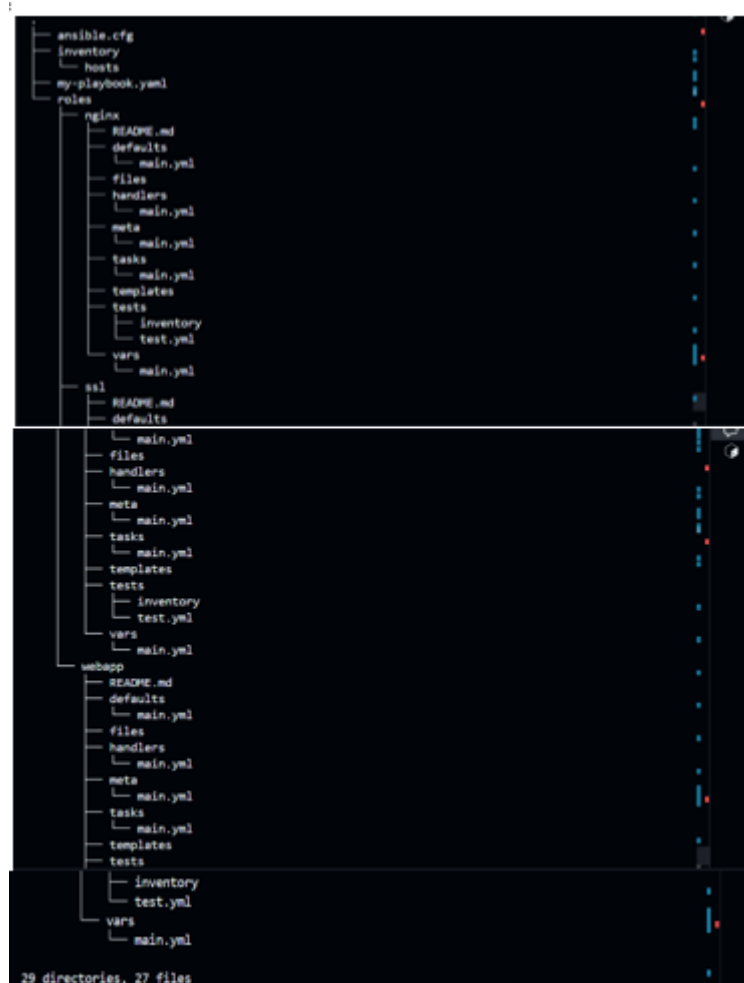
```
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3
```

The bottom file is `hosts` with the following content:

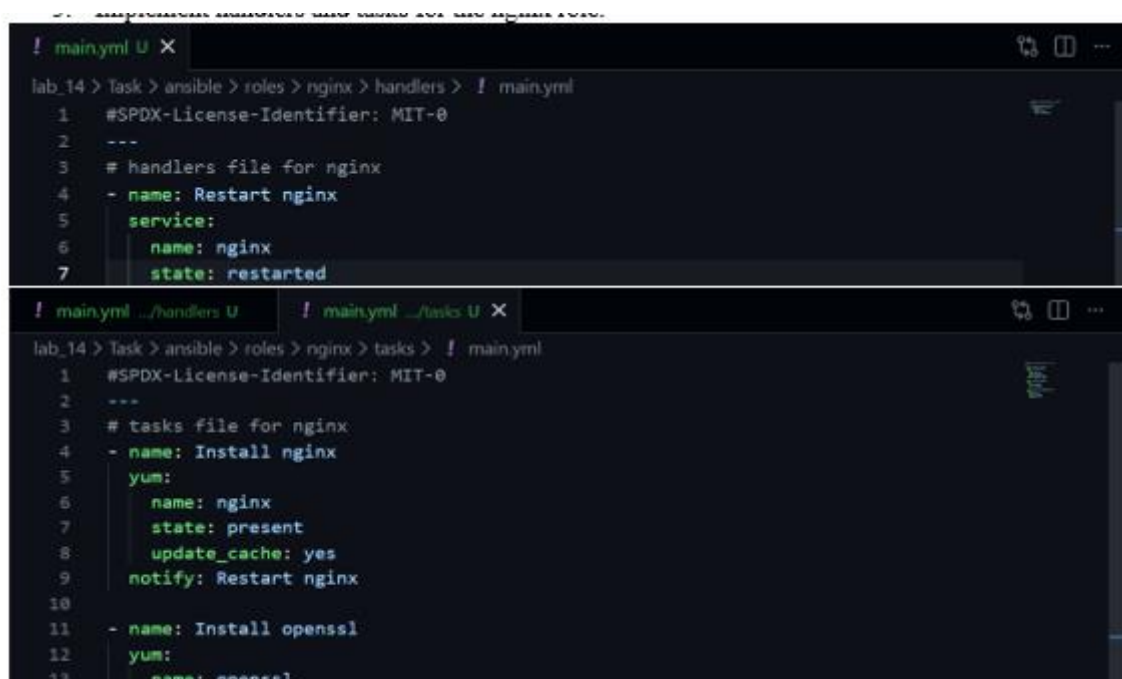
```
1 [nginx]
2 158.252.83.213
3
4 [nginx:vars]
5 ansible_ssh_private_key_file=~/.ssh/id_ed25519
6 ansible_user=ec2-user
```

Create roles for **nginx**, **ssl**, and **webapp** using ansible-galaxy.

```
Ansible-galaxy role init nginx, ansible-galaxy role init ssl, ansible-galaxy role init webapp, cd
tree
```



Implement handlers and tasks for the nginx role.



Test nginx role independently using a role-based playbook.

```
lab_14 > Task > ansible > ! my-playbook.yaml
1 ---
2 - name: Deploy NGINX Web Stack with SSL and PHP
3   hosts: nginx
4   become: true
5   roles:
6     - nginx
```

Ansible-playbook -I inventory/hosts my-playbook.yaml

```
TASK [nginx : Install openssl] *****
ok: [158.252.83.213]

TASK [nginx : Start and enable nginx] *****
ok: [158.252.83.213]

PLAY RECAP *****
158.252.83.213      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    resc
ued=0    ignored=0
```

Browser looks like this:

Initial Configuration

If you run Gitea inside Docker, please read the [documentation](#) before changing any settings.

Database Settings

Gitea requires MySQL, PostgreSQL, MSSQL, SQLite3 or TiDB (MySQL protocol).

Database Type *

Host *

Username *

Password *

Database Name *

SSL *

Schema

Leave blank for database default ("public")

Implement SSL role to generate self-signed certificates dynamically.

```
lab_14 > Task > ansible > roles > ssl > defaults > ! main.yml
1 #SPDX-License-Identifier: MIT-0
2 ---
3 # defaults file for ssl
4 imdsv2_token_ttl: "3600"
5 ssl_days_valid: 365

lab_14 > Task > ansible > roles > ssl > tasks > ! main.yml
1 #SPDX-License-Identifier: MIT-0
2 ---
3 # tasks file for ssl
4 - name: Create SSL private directory
5   file:
6     path: /etc/ssl/private
7     state: directory
8     mode: '0700'
```

Implement webapp role for PHP application deployment and Nginx templating.


```
lab_14 > Task > ansible > roles > webapp > defaults > ! main.yml
1 #SPDX-License-Identifier: MIT-0
2 ---
3 # defaults file for webapp
4 nginx_user: nginx
5 nginx_worker_processes: auto
6 nginx_worker_connections: 1024
7 nginx_error_log_level: notice
```

```
lab_14 > Task > ansible > roles > webapp > files > index.php
1 <?php
2 // Get hostname
3 $hostname = gethostname();
4
5 // Deployment date
6 $deployed_date = date("Y-m-d H:i:s");
7
8 // Metadata base URL
9 $metadata_base = "http://169.254.169.254/latest/";

lab_14 > Task > ansible > roles > webapp > handlers > ! main.yml
1 #SPDX-License-Identifier: MIT-0
2 ---
3 # handlers file for webapp
4 - name: Restart nginx
5   service:
6     name: nginx
7     state: restarted
8
9 - name: Restart php-fpm

lab_14 > Task > ansible > roles > webapp > templates > nginx.confj2
1 user {{ nginx_user }};
2 worker_processes {{ nginx_worker_processes }};
3 error_log /var/log/nginx/error.log {{ nginx_error_log_level }};
4 pid /run/nginx.pid;
5
6 events {
7     worker_connections {{ nginx_worker_connections }};
```

Create a final playbook combining all roles.

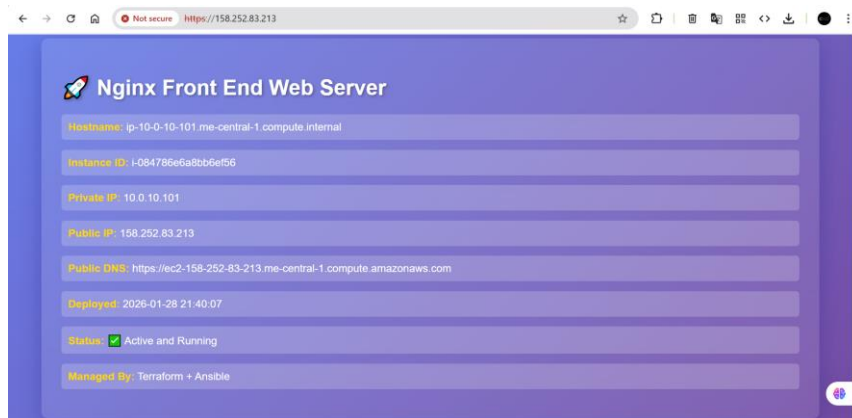
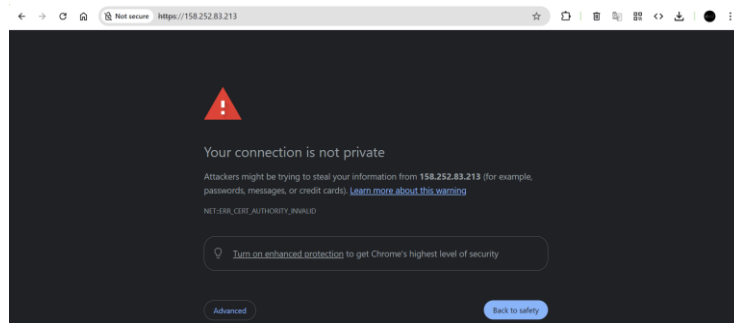
```
! my-playbook.yaml U X ! main.yml .../ssl/defaults U ! main.yml .../ssl/tasks U ! main.yml .../webapp U
lab_14 > Task > ansible > ! my-playbook.yaml
1 ---
2 - name: Deploy NGINX Web Stack with SSL and PHP
3   hosts: nginx
4   become: true
5   roles:
6     - nginx
7     - ssl
8     - webapp
```

Run the complete role-based playbook and verify HTTPS PHP application.: Ansible-playbook -I inventory/hosts my-playbook.yaml

```
TASK [webapp : Start and enable php-fpm] *****
ok: [158.252.83.213]

RUNNING HANDLER [webapp : Restart nginx] *****
changed: [158.252.83.213]

PLAY RECAP *****
158.252.83.213 : ok=15 changed=3 unreachable=0 failed=0 skipped=0 resc
ued=0 ignored=0
```



Cleanup – Destroy Resources & Verify State

Destroy all Terraform-managed resources using terraform destroy.

```
module.myapp-webserver-prod[0].aws_security_group.web_sg: Destroying... [id=sg-05bd3bea37989e92b]
module.myapp-webserver-prod[0].aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-subnet.aws_subnet.myapp_subnet_1: Destruction complete after 1s
module.myapp-webserver-prod[0].aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0c35f9c605b05e4c8]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 11 destroyed.
```

Verify Terraform state file shows no active resources: Cat terraform.tfstate

```
„check_liveness„: null
„resources„: [],
„outputs„: {},
„TJue986„: „q8epJPp-JT8J-pcqc-c3t2-28q82JPS9q6„,
„seuT9J„: „e“,
„f6669t0w„: „J‘J0‘0„,
„A6L2JOU„: „q“
{
```

Confirm via AWS Console that no EC2 instances are running.

